

TiRex: Zero-Shot Forecasting Across Long and Short Horizons

NXXAI



Sebastian Böck

Lead Researcher

sebastian.boeck@nx-ai.com

TiRex NeurIPS 2025:

<https://arxiv.org/abs/2505.23719>

GitHub:

<https://github.com/NX-AI/tirex>

Hugging Face:

<https://huggingface.co/NX-AI/TiRex>

arXiv:2505.23719v2 [cs.LG] 2 Nov 2025

TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning

Andreas Auer^{1,2}
Sebastian Böck¹

Patrick Podest²
Günter Klambauer^{1,2}

Daniel Klotz³
Sepp Hochreiter^{1,2}



¹NXAI GmbH, Linz, Austria

²ELLIS Unit, LIT AI Lab, Institute for Machine Learning, JKU Linz, Austria

³Interdisciplinary Transformation University Austria, Linz, Austria

Abstract

In-context learning, the ability of large language models to perform tasks using only examples provided in the prompt, has recently been adapted for time series forecasting. This paradigm enables zero-shot prediction, where past values serve as context for forecasting future values, making powerful forecasting tools accessible to non-experts and increasing the performance when training data are scarce. Most existing zero-shot forecasting approaches rely on transformer architectures, which, despite their success in language, often fall short of expectations in time series forecasting, where recurrent models like LSTMs frequently have the edge. Conversely, while LSTMs are well-suited for time series modeling due to their state-tracking capabilities, they lack strong in-context learning abilities. We introduce TiRex that closes this gap by leveraging xLSTM, an enhanced LSTM with competitive in-context learning skills. Unlike transformers, state-space models, or parallelizable RNNs such as RWKV, TiRex retains state-tracking, a critical property for long-horizon forecasting. To further facilitate its state-tracking ability, we propose a training-time masking strategy called CPM. TiRex sets a new state of the art in zero-shot time series forecasting on the HuggingFace benchmarks *GiftEval* and *Chronos-ZS*, outperforming significantly larger models including *TabPFN-TS* (Prior Labs), *Chronos Bolt* (Amazon), *TimesFM* (Google), and *Moirai* (Salesforce) across both short- and long-term forecasts.

1 Introduction

Recent research in time series forecasting has adopted in-context learning through large-scale pre-trained models, analogous to large language models (Woo et al., 2024; Ansari et al., 2024a; Das et al., 2024). These models enable zero-shot forecasting, allowing them to generalize to unseen datasets without parameter updates, akin to meta-learning Hochreiter et al. (2001). This capability empowers practitioners without machine learning expertise to use advanced forecasting tools. More importantly, zero-shot forecasting significantly improves performance in data-scarce settings, where training task-specific models often fail to generalize. As a result, in-context learning models hold promise for broad adoption in domains such as energy, retail, or healthcare.

Most pre-trained time series models are based on transformer architectures (Vaswani et al., 2017), which are well suited for in-context learning, but despite their success in language, often fall short of expectations in time series forecasting (e.g., Zeng et al., 2023). In contrast, LSTMs (Hochreiter, 1991; Hochreiter & Schmidhuber, 1997) have demonstrated strong results in time series forecasting due to their recurrence and effective state-tracking (e.g., Neuring et al., 2024). Therefore, LSTMs are more expressive than state-space models (SSMs), parallelizable RNNs like RWKV (Peng et al., 2023), and

Zero-Shot Forecasting with Pre-Trained Models

Forecast future timesteps of time series given its historical context

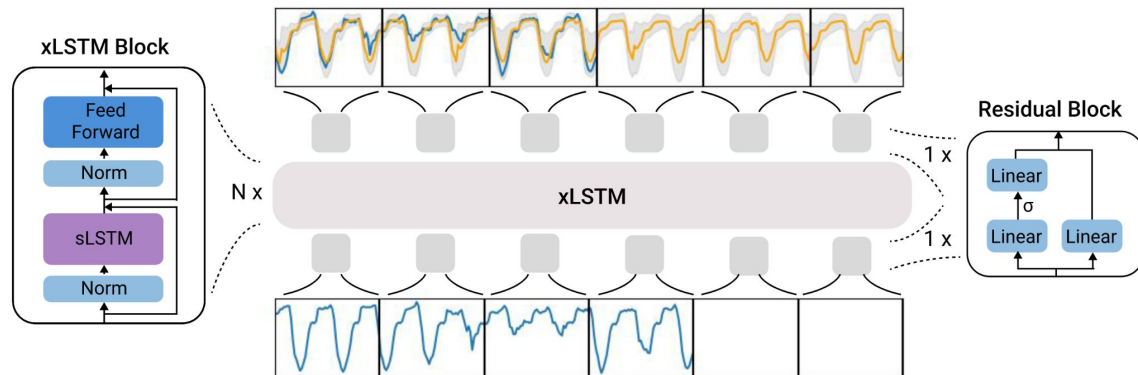
$$\mathcal{P}(\mathbf{y}_{T+1:T+h} \mid \mathbf{y}_{1:T})$$

Pre-trained forecasting models provide forecasts for unknown datasets (zero-shot)

- ✓ Competitive forecasting in zero-shot mode
- ✓ Accessible forecasts without modeling or machine learning knowledge

TiRex Architecture

- Decoder-only architecture
 - 35 M parameters
- Input Layer via residual block
 - Instance normalization
 - Patch size: 32
 - Context length: 2048
- xLSTM backbone
 - sLSTM cells
- Output Layer via residual block
 - 10 quantiles per time step



Transformers and their limits

Vaswani et al, "Attention Is All You Need", NeurIPS 2017

Enabled:

✓ Massive scaling through parallelizable self-attention

Abandoned:

✗ State tracking capability

Introduced:

✗ Quadratic cost $O(n^2)$ and linearly growing KV cache

→ We need parallelizability with state-tracking and linear-time inference

xLSTM

xLSTM NeurIPS 2024:

<https://arxiv.org/abs/2405.04517>

xLSTM 7B ICML 2025:

<https://arxiv.org/abs/2503.13427>

GitHub:

<https://github.com/NX-AI/xlstm>

Hugging Face:

<https://huggingface.co/NX-AI/xLSTM-7b>

arXiv:2405.04517v2 [cs.LG] 6 Dec 2024

xLSTM: Extended Long Short-Term Memory

Maximilian Beck*^{1,2,3} Korbrian Pöppel*^{1,2,3} Markus Spanring¹
 Andreas Auer^{1,2} Oleksandra Prudnikova¹ Michael Kopp
 Günter Klambauer^{1,2,3} Johannes Brandstetter^{1,2,3} Sepp Hochreiter^{1,2,3}

*Equal contribution

¹ELLIS Unit, LIT AI Lab, Institute for Machine Learning, JKU Linz, Austria

²NXAI Lab, Linz, Austria, ³NXAI GmbH, Linz, Austria

Abstract

In the 1990s, the constant error carousel and gating were introduced as the central ideas of the Long Short-Term Memory (LSTM). Since then, LSTMs have stood the test of time and contributed to numerous deep learning success stories, in particular they constituted the first Large Language Models (LLMs). However, the advent of the Transformer technology with parallelizable self-attention at its core marked the dawn of a new era, outpacing LSTMs at scale. We now raise a simple question: How far do we get in language modeling when scaling LSTMs to billions of parameters, leveraging the latest techniques from modern LLMs, but mitigating known limitations of LSTMs? Firstly, we introduce exponential gating with appropriate normalization and stabilization techniques. Secondly, we modify the LSTM memory structure, obtaining: (i) sLSTM with a scalar memory, a scalar update, and new memory mixing, (ii) mLSTM that is fully parallelizable with a matrix memory and a covariance update rule. Integrating these LSTM extensions into residual block backbones yields xLSTM blocks that are then residually stacked into xLSTM architectures. Exponential gating and modified memory structures boost xLSTM capabilities to perform favorably when compared to state-of-the-art Transformers and State Space Models, both in performance and scaling. Code available at: <https://github.com/NX-AI/xlstm>

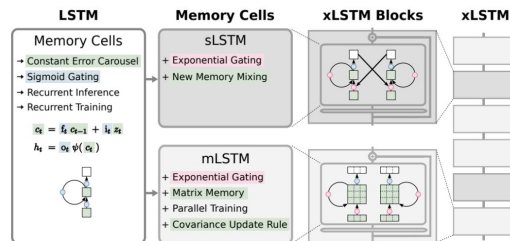


Figure 1: The extended LSTM (xLSTM) family. From left to right: 1. The original LSTM memory cell with constant error carousel and gating. 2. New sLSTM and mLSTM memory cells that introduce exponential gating. sLSTM offers a new memory mixing technique. mLSTM is fully parallelizable with a novel matrix memory cell state and new covariance update rule. 3. mLSTM and sLSTM in residual blocks yield xLSTM blocks. 4. Stacked xLSTM blocks give an xLSTM architecture.



LSTM → xLSTM

Step 1: The vanilla LSTM fails at scale

Model	Modification	Exp. Gating	Matrix Mem.	#Params M	SlimPajama ppl ↓
LSTM	Vanilla Multi-Layer LSTM	✗	✗	607.8	2417.86
LSTM	+ ResNet Backbone	✗	✗	506.1	35.46
LSTM	+ Up-Projection Backbone	✗	✗	505.9	26.01
xLSTM[0:1]	+ Exponential Gating	✓	✗	427.3	17.70
xLSTM[7:1]	+ Matrix Memory	✓	✓	408.4	13.48

LSTM → xLSTM

Step 2: ResNet backbone

Model	Modification	Exp. Gating	Matrix Mem.	#Params M	SlimPajama ppl ↓
LSTM	Vanilla Multi-Layer LSTM	✗	✗	607.8	2417.86
LSTM	+ ResNet Backbone	✗	✗	506.1	35.46
LSTM	+ Up-Projection Backbone	✗	✗	505.9	26.01
xLSTM[0:1]	+ Exponential Gating	✓	✗	427.3	17.70
xLSTM[7:1]	+ Matrix Memory	✓	✓	408.4	13.48

LSTM → xLSTM

Step 3: Up-projection backbone

Model	Modification	Exp. Gating	Matrix Mem.	#Params M	SlimPajama ppl ↓
LSTM	Vanilla Multi-Layer LSTM	✗	✗	607.8	2417.86
LSTM	+ ResNet Backbone	✗	✗	506.1	35.46
LSTM	+ Up-Projection Backbone	✗	✗	505.9	26.01
xLSTM[0:1]	+ Exponential Gating	✓	✗	427.3	17.70
xLSTM[7:1]	+ Matrix Memory	✓	✓	408.4	13.48

LSTM → xLSTM

Step 4: Exponential gating → the sLSTM

Model	Modification	Exp. Gating	Matrix Mem.	#Params M	SlimPajama ppl ↓
LSTM	Vanilla Multi-Layer LSTM	✗	✗	607.8	2417.86
LSTM	+ ResNet Backbone	✗	✗	506.1	35.46
LSTM	+ Up-Projection Backbone	✗	✗	505.9	26.01
xLSTM[0:1]	+ Exponential Gating	✓	✗	427.3	17.70
xLSTM[7:1]	+ Matrix Memory	✓	✓	408.4	13.48

LSTM → xLSTM

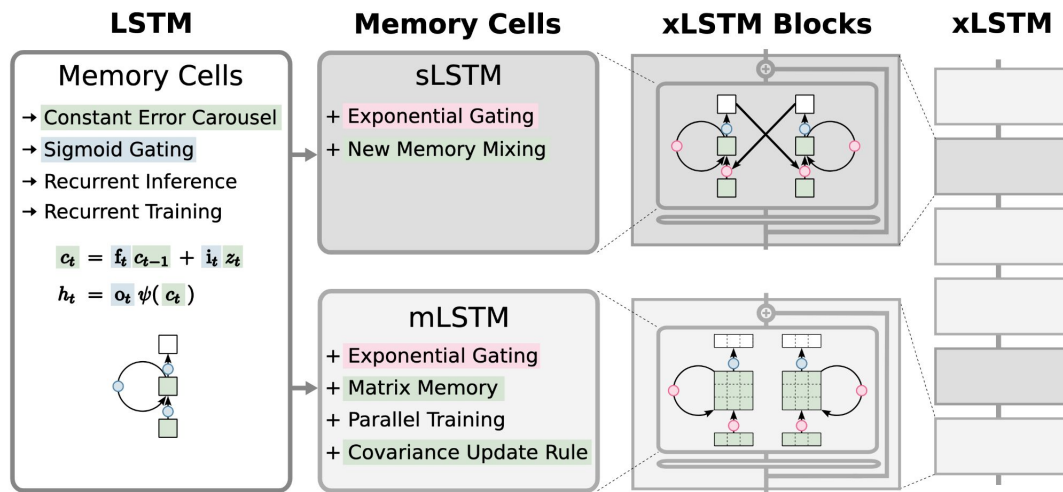
Step 5: Matrix memory → the mLSTM

Model	Modification	Exp. Gating	Matrix Mem.	#Params M	SlimPajama ppl ↓
LSTM	Vanilla Multi-Layer LSTM	✗	✗	607.8	2417.86
LSTM	+ ResNet Backbone	✗	✗	506.1	35.46
LSTM	+ Up-Projection Backbone	✗	✗	505.9	26.01
xLSTM[0:1]	+ Exponential Gating	✓	✗	427.3	17.70
xLSTM[7:1]	+ Matrix Memory	✓	✓	408.4	13.48

xLSTM: Extended Long Short-Term Memory

xLSTM puts LSTM into the modern era, addressing its shortcomings:

1. Inability to revise storage decisions
→ Exponential gating
2. Limited storage capacity
→ Matrix Memory
3. Efficiency: Lack of parallelizability
→ New Memory Mixing
→ Fully parallelizable variant



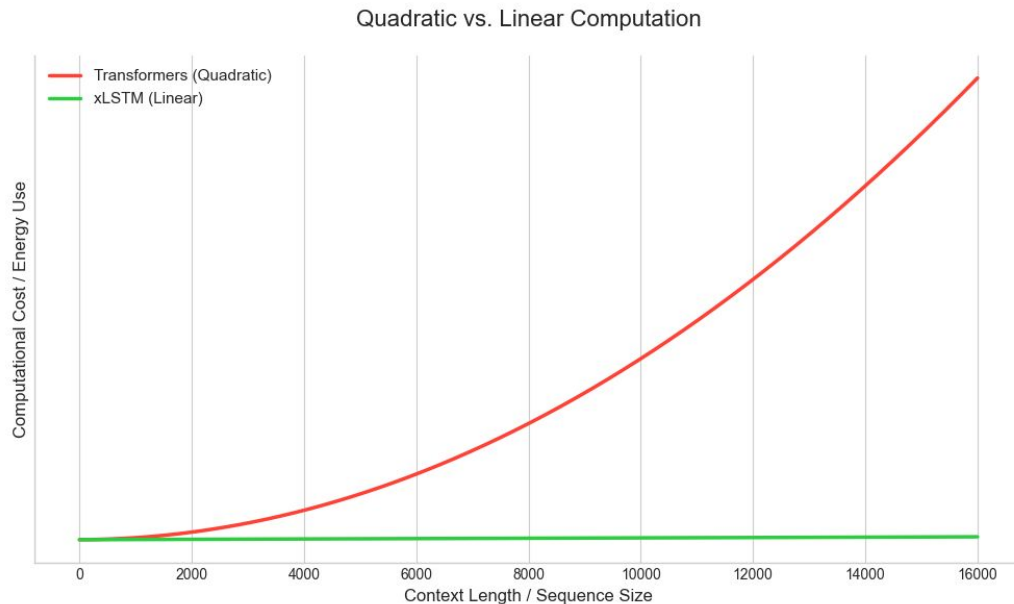
xLSTM: Linear Computation & Constant Memory

The primary benefit:

- linear computational complexity
- constant memory complexity
- advanced in-context learning capabilities
- energy efficiency

Ideal for modeling long-range dependencies in

- language
- vision
- time series



xLSTM Backbone

TiRex utilises sLTMS cells:

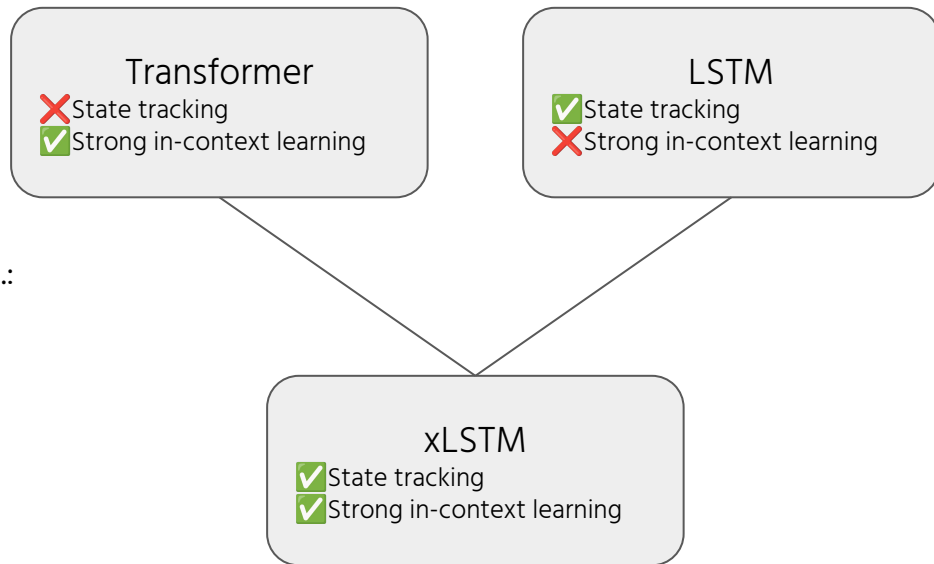
- Support state tracking
- Enhanced in-context learning

State tracking tasks cannot be solved by Transformers, e.g.:

- Parity computations
- Modulus arithmetics

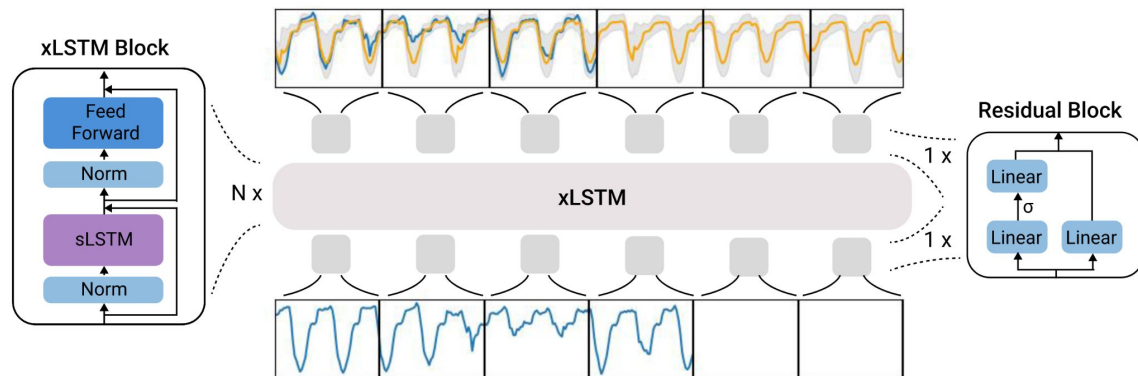
In-context learning:

- Ability to adapt to new tasks or context by simply providing examples as input, without requiring any model updates or retraining



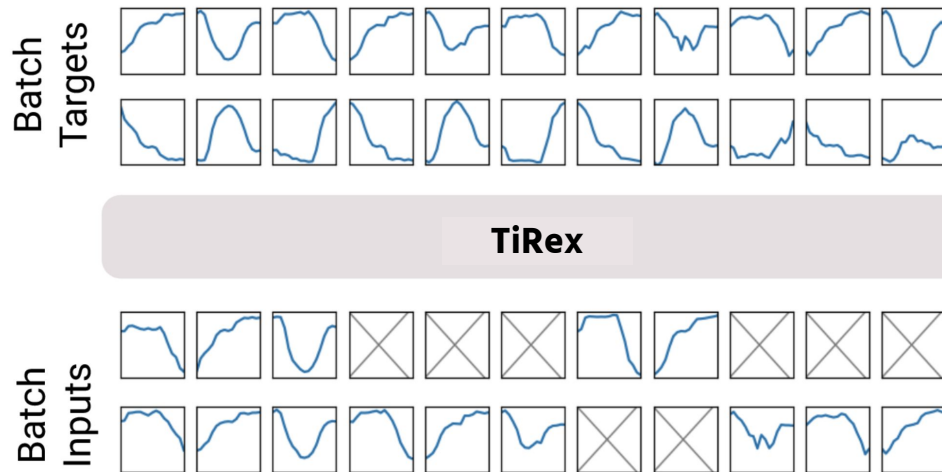
TiRex Overview

- Decoder-only architecture
 - 35 M parameters
- Input Layer via residual block
 - Patch size: 32 samples
 - Instance normalization
 - Context length: 2048
- xLSTM backbone
 - sLSTM cells
- Output Layer via residual block
 - 10 quantiles per time step



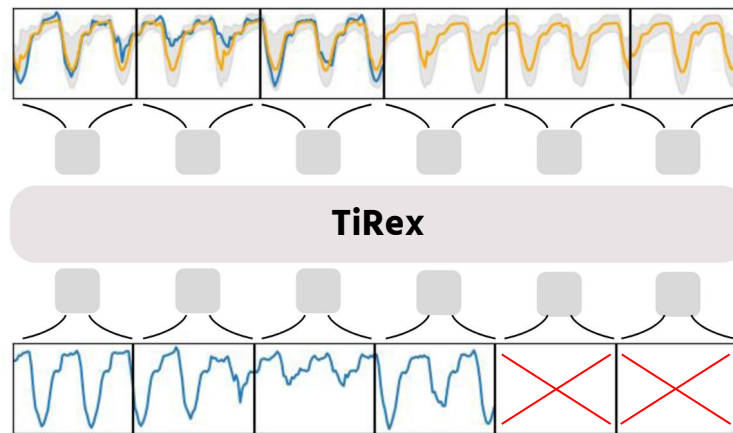
Contiguous Patch Masking (CPM)

- Decoder-only training with causal modeling
- Mask out consecutive patches
- TiRex is forced to propagate information over the cell state



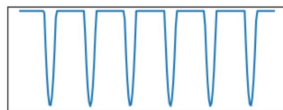
Masked Inference

- Use masked input when forecasting future tokens
- Advantages:
 - No “uncertainty collapse” as with autoregressive input
 - More efficient than Monte Carlo sampling
 - No model output needed as input for training

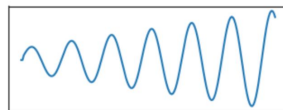


Training Augmentations

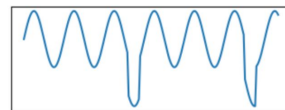
- Stochastically augment samples during training
 - Censor Augmentation
 - Amplitude Modulation
 - Spike Injection
- Diverse training data + exposure to potentially relevant dynamics
 - Chronos Training Data + adapted TsMixup
 - Synthetic Gaussian Process Data
 - GiftEval Pre-training Data (Subset)
 - ~50M sequences in total



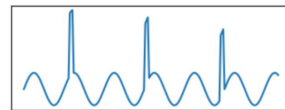
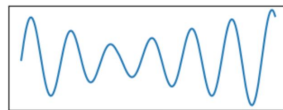
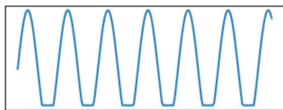
Censor Augmentation



Amplitude Modulation



Spike Injection



Results

fev-bench

🏆 Leaderboard

Results for various forecasting models on 100 tasks of the **fev-bench** benchmark, as described in the paper [fev-bench: A Realistic Benchmark for Time Series Forecasting](#).

Model Name	Avg. win rate (%)	Skill score (%)	Median runtime (s)	Leakage (%)	Failed tasks (%)	Zero-shot	Organization	Link
Chronos-2	91.4	47.3	3.6	0	0	✓	AWS	🔗
TiRex	82.7	42.6	1.4	1	0	✓	NX-AI	🔗
TimesFM-2.5	77.6	42.2	16.9	10	0	✓	Google	🔗
Toto-1.0	70.2	40.7	90.7	8	0	✓	Datadog	🔗
Chronos-Bolt	64.4	38.9	1.0	0	0	✓	AWS	🔗
Moirai-2.0	64.4	39.3	2.5	28	0	✓	Salesforce	🔗
TabPFN-TS	63.0	39.6	305.5	0	2	✓	Prior Labs	🔗
Sundial-Base	46.0	33.4	35.6	1	0	✓	Tsinghua University	🔗
Stat. Ensemble	45.5	20.2	690.6	0	11	✗	—	🔗
AutoARIMA	40.7	20.6	186.8	0	10	✗	—	🔗

Results

GIFT-Eval Time Series Forecasting Leaderboard

We introduce the General Time Series Forecasting Model Evaluation, GIFT-Eval, a pioneering benchmark aimed at promoting evaluation across diverse datasets. GIFT-Eval encompasses 24 datasets over 144,000 time series and 177 million data points, spanning seven domains, 10 frequencies, multivariate inputs, and prediction lengths ranging from short to long-term forecasts.

Overall

By Domain

By Frequency

By Term Length

By Variate Type

About

Search

Separate multiple queries with `^`

Select Columns to Display:

Organization

Test Leak.

Replication Code

MASE

MASE_Rank

CRPS

CRPS_Rank

Model types

zero-shot

agentic

pretrained

deep-learning

statistical

fine-tuned

Test Leak.

No

Yes

Replication Code

No

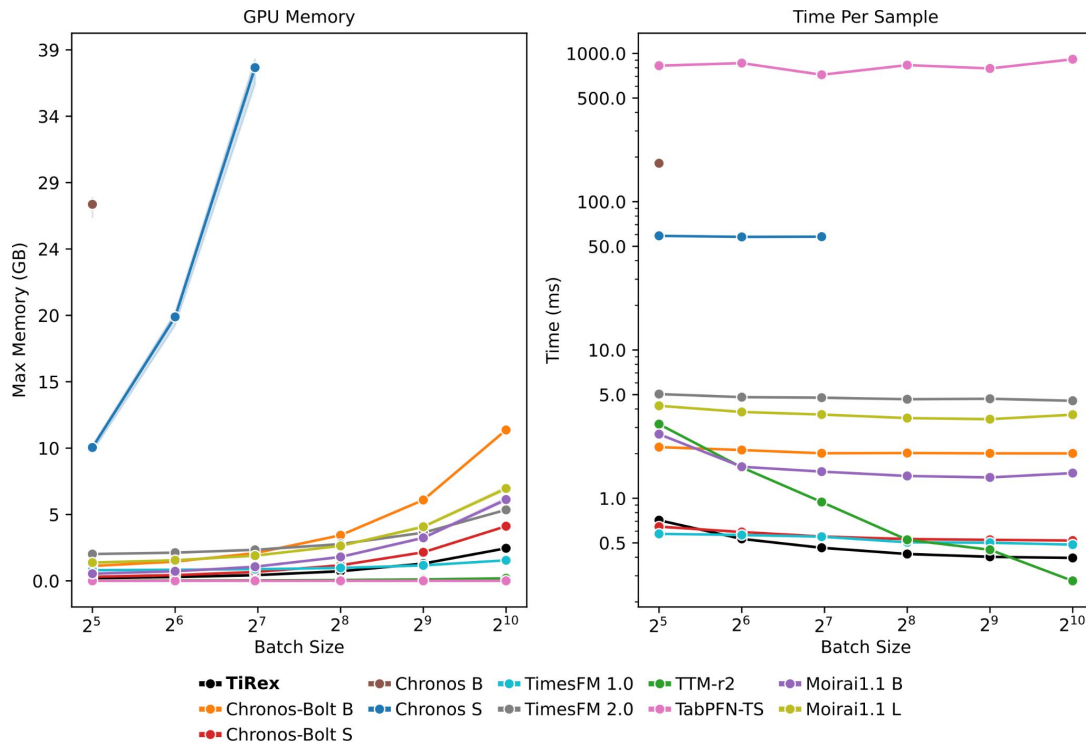
Yes

Show Rows with the Following Values

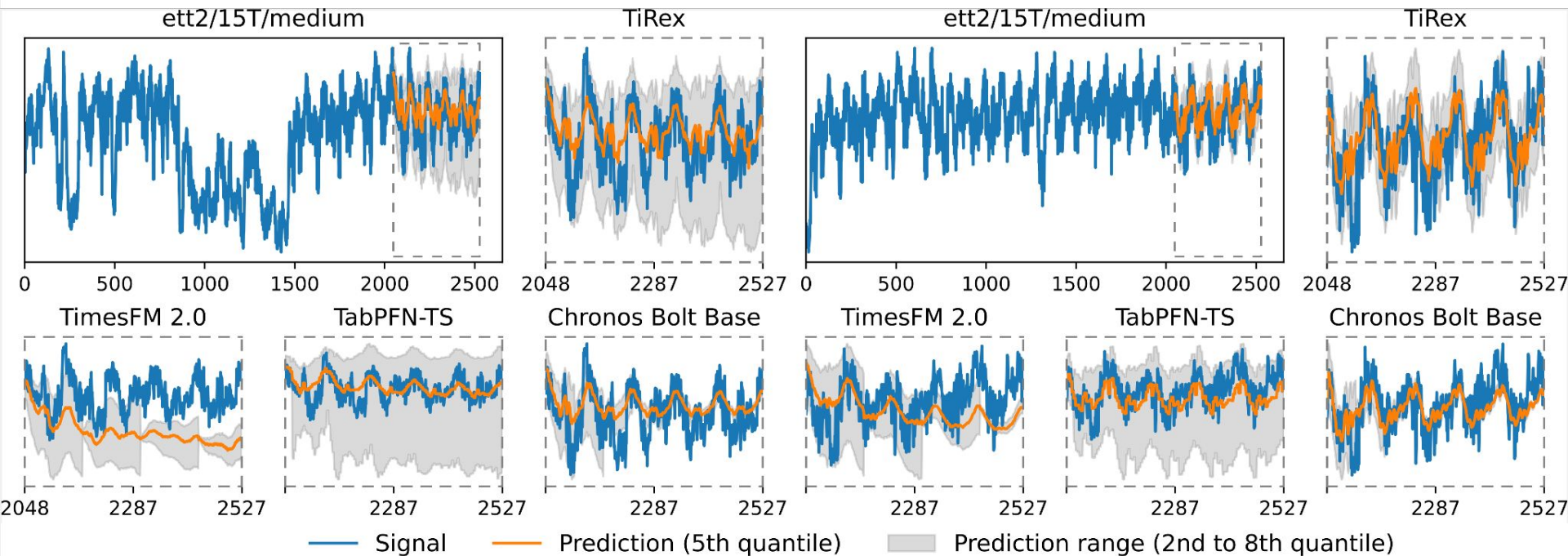
T	model	Organization	Test Leak.	Replication Code	MASE	MASE_Rank	CRPS	CRPS_Rank
	TiRex (code)	NXAI	No	Yes	0.716	12.464	0.488	10.577
	Chronos-2 (code)	AWS	No	Yes	0.698	10.186	0.485	10.804
	TimesFM-2.5 (code)	Google Research	No	Yes	0.705	11.856	0.49	12.134
	FlowState-2.1M (code)	IBM Research	No	Yes	0.726	16.33	0.502	14.402
	granite-flowstate-r1 (code)	IBM Research	No	Yes	0.732	17.464	0.505	15.557
	Toto_Open_Base_1.0 (code)	Datadog	No	Yes	0.750	18.938	0.517	16.979
	Moirai2 (code)	Salesforce AI Resea	No	Yes	0.728	17.32	0.516	17.948
	YingLong_300m (code)	Alibaba	No	Yes	0.798	23.835	0.548	21.433

Inference Efficiency & Speed

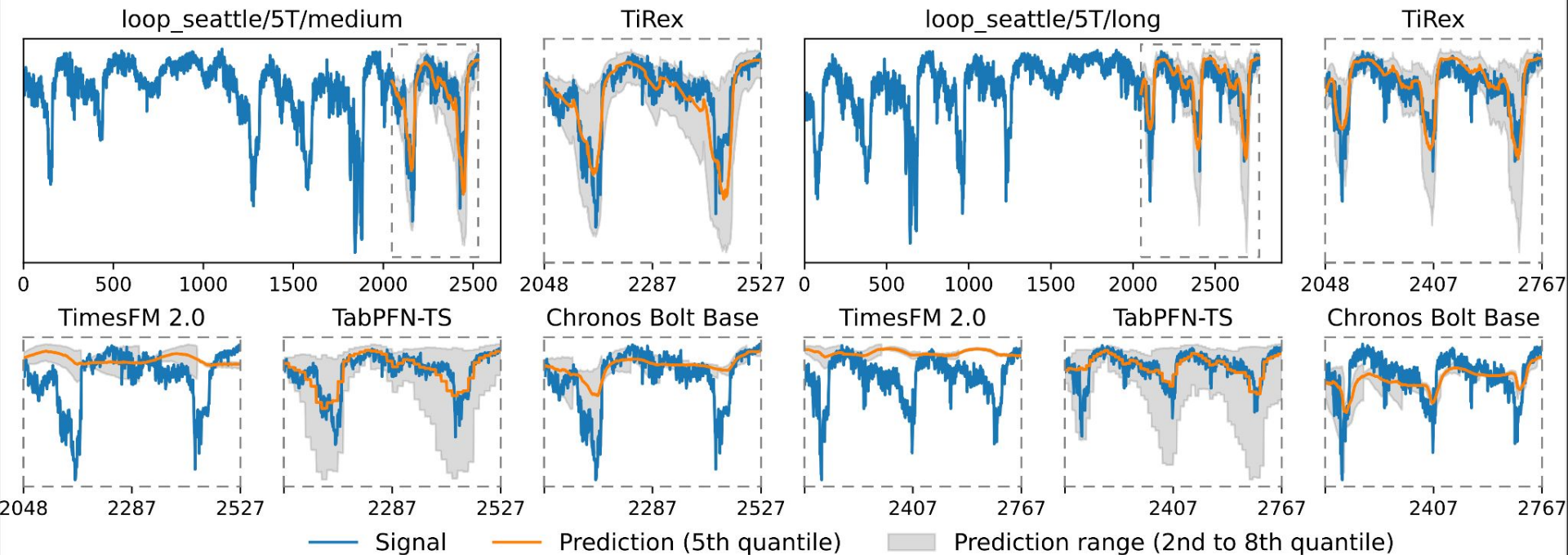
- Among fastest models overall
- Fast sLSTM kernels
- Only 35M parameters
- Considerably faster than SOTA
- Low GPU memory footprint



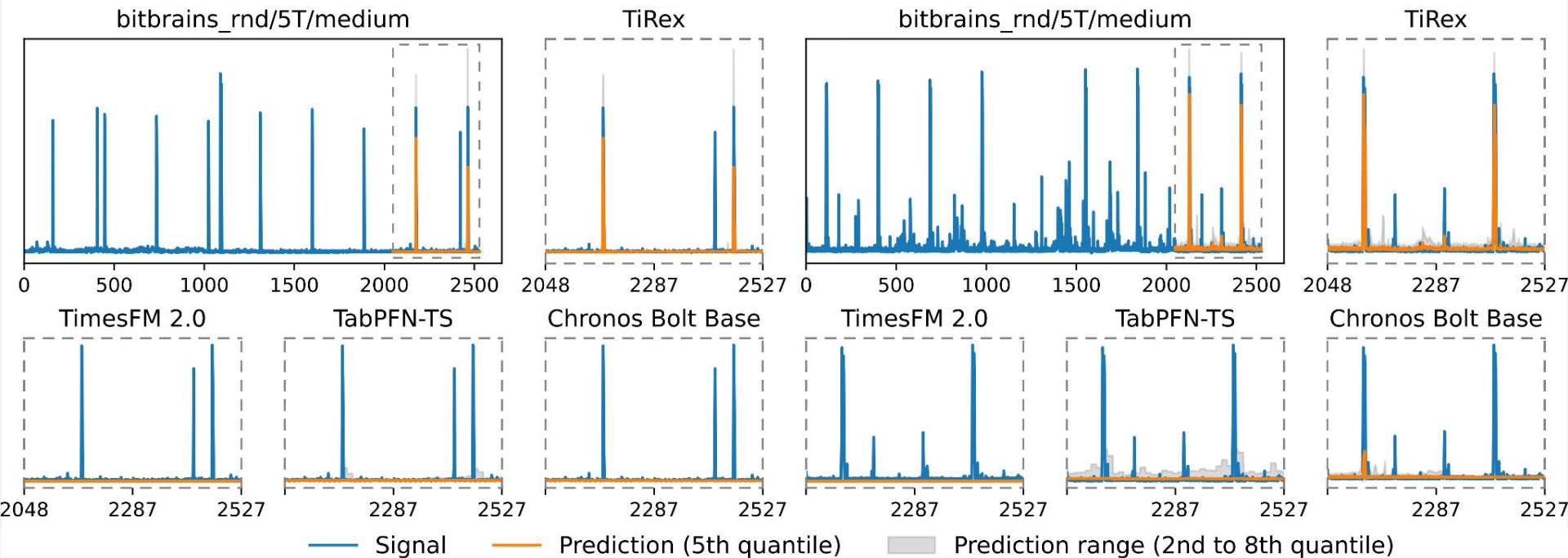
Forecasting Examples



Forecasting Examples



Forecasting Examples



Applications

TiRex for time series classification:

<https://arxiv.org/abs/2510.26777>

arXiv:2510.26777v1 [cs.LG] 30 Oct 2025

Pre-trained Forecasting Models: Strong Zero-Shot Feature Extractors for Time Series Classification

Andreas Auer^{1,2} Daniel Klotz³ Sebastian Böck¹ Sepp Hochreiter^{1,2}

¹NXA GmbH, Linz, Austria

²ELLIS Unit, LIT AI Lab, Institute for Machine Learning, JKU Linz, Austria

³Interdisciplinary Transformation University Austria, Linz, Austria

Abstract

Recent research on time series foundation models has primarily focused on forecasting, leaving it unclear how generalizable their learned representations are. In this study, we examine whether frozen pre-trained forecasting models can provide effective representations for classification. To this end, we compare different representation extraction strategies and introduce two model-agnostic embedding augmentations. Our experiments show that the best forecasting models achieve classification accuracy that matches or even surpasses that of state-of-the-art models pre-trained specifically for classification. Moreover, we observe a positive correlation between forecasting and classification performance. These findings challenge the assumption that task-specific pre-training is necessary, and suggest that learning to forecast may provide a powerful route toward constructing general-purpose time series foundation models.

1 Introduction

In time series forecasting, foundation models are becoming increasingly prominent. They are large models that are pre-trained on broad data, and therefore have the ability to generalize across unseen datasets [3, 33, 14, 13, 6]. New benchmarks with public leaderboards such as GiftEval [1] and BOOM [13] have accelerated advances in state-of-the-art methods. Apart from forecasting, Time Series Classification (TSC) is another key application in time series analysis.

Earlier general-purpose time series models [20, 19] evaluated multiple downstream tasks, but recent work shows that they have failed to reach state-of-the-art performance in either forecasting or classification [17, 16]. More recently, the majority of newly introduced foundation models [6, 13, 25, 32, 22] have been optimized specifically for forecasting, and only few have focused on classification [17, 24]. Some argue that pre-training objectives should be aligned with downstream applications, for example, contrastive objectives for classification or masked reconstruction for imputation [17]. This perspective suggests that task-specialized pre-training may be necessary for optimal performance, which is in contrast to language and vision foundation models, where a single pre-trained model often transfers effectively across many diverse tasks [11, 12].

This contrast motivates our central research question: **How well do representations from pre-trained forecasting models transfer to classification tasks?** To answer this question, we evaluate a diverse set of forecasting models as frozen feature extractors on TSC benchmarks, analyze key design choices for representation extraction, and investigate the role of model architectures. Beyond the direct application to classification, our study aims to provide broader insights into the generalizability of learned representations, which is a step toward developing true time series foundation models.

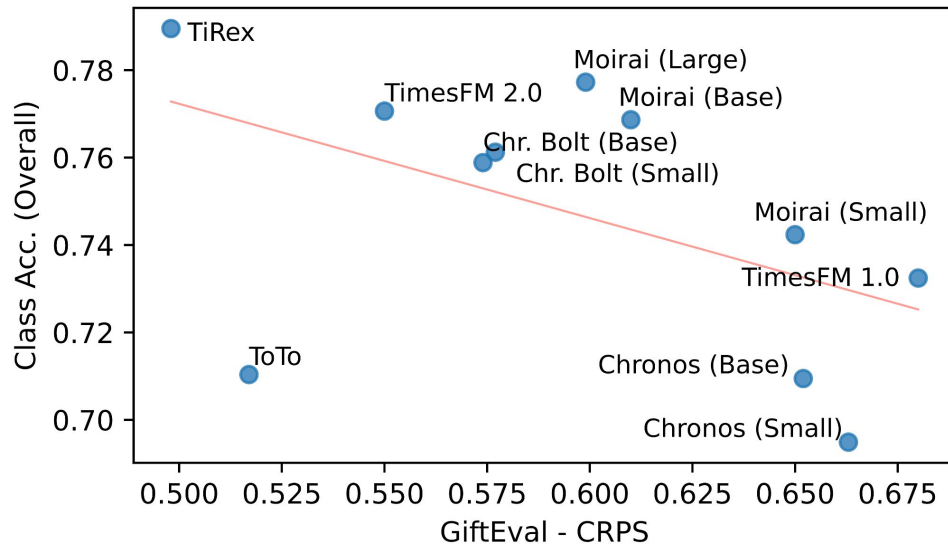
Our contributions are as follows: (1) We show that representations from pre-trained forecasting models yield classification accuracy on par with, and in some cases surpassing state-of-the-art

Applications

Time Series Classification

Use the internal representation of TiRex to classify time series:

- Mean Pooling over sequence dimension and concatenation of layer-wise representations
- Classification with Random Forest
- Better forecasting models yield better classification accuracies



Applications

Time Series Classification

- TiRex yields classification accuracy on par with, and in some cases surpassing state-of-the-art models pre-trained explicitly for classification
- Classification accuracy can be improved further by incorporating absolute statistical features and differentiated series

	Type	ZS	Univariate		Multivariate		Overall	
			No Aug	Stat+Diff	No Aug	Stat+Diff	No Aug	Stat+Diff
TiRex	Dec	yes	0.80	0.81	0.74	0.74	0.79	0.80
Chr. Bolt (Base)	EncDec	yes	0.77	0.79	0.72	0.74	0.76	0.78
Moirai (Large)	Enc	yes	0.79	0.80	0.70	0.70	0.78	0.78
TimesFM 2.0	Dec	yes	0.79	0.79	0.70	0.70	0.77	0.78
TimesFM 1.0	Dec	yes	0.74	0.75	0.71	0.72	0.73	0.74
Chronos (Base)	EncDec	yes	0.71	0.76	0.71	0.72	0.71	0.75
Toto	Dec	yes	0.71	0.74	0.71	0.70	0.71	0.73

Mantis	Enc	no		0.79		0.74		0.78
NuTime	Enc	no		0.67		0.68		0.67
Moment (Large)	Enc	no		0.63		0.57		0.62
DTW	-	-		0.73		0.72		0.73

Conclusions

- TiRex utilises advantages of xLSTM:
State-tracking + strong in context learning
- Contiguous Patch Masking + Masked Inference:
Enables fast long-term uncertainty estimates without sacrificing short-term performance
- Effective Data Augmentations:
Censor, Spike, Amplitude Modulation
- SOTA in zero-shot forecasting at public leaderboards
- Very fast and low GPU memory footprint
- Internal representation well suited for downstream tasks (e.g. classification)
No need to train specialised models for classification tasks?

Future (current) work: TiRex 2

- Extended data handling:
 - Multivariates
 - Past & future covariates
 - Exogenous data
- Fine-tuning towards special domains
- Longer forecast horizons
- More downstream tasks:
 - Anomaly detection
 - Vector-representation of time series (e.g. for similarity search)

Thank you!

References

- xLSTM: Extended Long Short-Term Memory
(NeurIPS 2024. <https://arxiv.org/abs/2405.04517>)
- TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning
(NeurIPS 2025. <https://arxiv.org/abs/2505.23719>)
- Pre-trained Forecasting Models: Strong Zero-Shot Feature Extractors for Time Series Classification
(NeurIPS 2025 BERT2S workshop. <https://openreview.net/pdf?id=vhngjDHvB0>)

NXXAI